Guide d'intégration BO6 — M7 « Catalogue local » et M14 « Stylist / boutique mode »

Pour la personne qui intègre les modules DressMe. Rédigé le 21/10/2026, mis à jour le 24/10/2026 par BO6 (kenzabokri). Branche : `BO6/local_catalogue`.

---

0. En bref

	M7 Catalogue local	M14 Stylist / boutique mode
Rôle	Chercher des pièces dans 43 boutiques tunisiennes (6 375 produits, prix en TND)	Côté professionnel : boutiques et stylistes, tableau de bord de la demande, hand-off assistant → styliste, conversation privée, looks
Agent	`localshop` = l'agent BO6 (un agent par BO). `tool.py` trouvé par `orchestrator/registry.py`	**Pas d'agent** : compétence `boutique` de `localshop`. Pas de `tool.py`
État	Réel et complet (données, FashionCLIP, évaluation, mémoire, `next_actions`) ; **M1 et M2 simulés** pour les idées 3, 4 et 10 tant qu'ils ne sont pas fusionnés	Réel pour M7 ; **M1, M2, M3, M5, M6, M10 sur mock** tant qu'ils ne sont pas fusionnés
LLM	**Qwen 2.5 7B en local (Ollama)**, un seul pour M7 et M14 (`m07_local_catalogue/llm.py`) : comprend les requêtes difficiles	Le même LLM : résumé pour la styliste, escalade, conseils boutique
Tests	`pytest modules/m07_local_catalogue` (197)	`pytest modules/m14_stylist_mode` (51)
App	`/#/bo6_local_market`	`/#/bo6_boutique` (Espace pro) et `/#/bo6_boutique/help/:pro/:client`

Ce qu'il reste à faire pour M14, par priorité (détails au §4) : 1. Un **identifiant utilisateur commun** à tous les modules, puis l'**authentification** (JWT). 2. Passer **M1, M3, M2, M6, M10** en réel : un seul changement dans `config.yaml`. 3. **M5** : se mettre d'accord avec BO4 sur le format du profil (M5 n'existe pas encore). 4. **Orchestrateur (BO8)** : le bouton « Parler à un styliste » dans le chat → `POST /m14/handoff`. 5. Remplacer le stockage JSON par **Postgres**. 6. **App côté client** : consentement, looks reçus, conversation. 7. **LLM** : installer Ollama et `qwen2.5:7b` sur la machine qui fait tourner l'API (§4.11). Sans lui, tout marche quand même (règles).

---

0 bis. Ce que le BO6 a préparé pour faciliter l'intégration

L'idée générale : **chaque appel vers un autre module est déjà écrit au bon format**. Tant que ce module n'est pas fusionné, une simulation répond à sa place. Le jour de la fusion, on change **une ligne de configuration** ; le code de M7 et de M14 ne bouge pas.

Ce qui est prêt	Où	Ce que ça évite à l'intégrateur
Un interrupteur par module : `real: [...]` dans `config.yaml` (M7 et M14)	`m07_local_catalogue/config.yaml`, `m14_stylist_mode/config.yaml`	Pas de code à modifier : on ajoute `m1`, `m2`, `m10`… à la liste après chaque fusion
Repli automatique : un module listé dans `real` qui ne répond pas → la simulation reprend	`clients.py → _try` (M7 et M14)	Une branche en panne ne casse pas le BO6 ; l'app continue de marcher
`sources` dans chaque réponse : `{"m1": "real", "m2": "mock"}`	sorties de `/m07/search`, `/m07/outfits`, `/m07/local-pieces`, fiches et looks M14	On voit tout de suite quel module est vraiment branché
Formats validés sur les vraies branches : M2 (`/score`, `/complete`, format `Garment`, vecteur 1024), M10 (`CardItem`), M6 (`SmartBuyInput`), M1 (`/m01/items`)	`clients.py` (M7 et M14)	Pas de surprise de format au moment de la fusion
Le chemin « réel » déjà testé avec de faux modules (bonne réponse, puis panne → simulation)	`m07_local_catalogue/tests/test_wardrobe.py`, tests M14	La bascule est vérifiée avant même la fusion
`mock.py` pour les autres BO : réponses fixes au format exact de chaque endpoint (`search`, `shops`, `outfits`, `local-pieces` ; tout M14)	`m07_local_catalogue/mock.py`, `m14_stylist_mode/mock.py` (testés par `test_mock.py`)	Les autres BO peuvent coder contre le BO6 sans le catalogue ni les modèles
Contrats Pydantic typés, champs nouveaux toujours **optionnels**	`contract.py` (M7 et M14)	Smart Buy et BO7, qui appellent déjà M7, ne cassent pas
Carte d'agent (`GET /m07/card`) et `tool.py` trouvé automatiquement par `orchestrator/registry.py` ; la description de l'outil est générée depuis la carte	`identity.py`, `tool.py`	Rien à enregistrer à la main dans l'orchestrateur
`next_actions` au lieu d'appels entre agents : `ask_user`, `open_shop`, `judge` → smartbuy, `show_outfits` → stylist, `try_on` → tryon, `more_results`, `feedback`	`agent.py → _next_actions`	L'orchestrateur (BO8) chaîne les agents ; aucun agent n'appelle un autre agent (règle du guide des agents)
Mémoire au format du guide : `session_id`, `answer`, `next_page`, `decision_id`, `DELETE /m07/memory/{user}`	`memory.py`	Même forme que Smart Buy ; le droit à l'oubli existe déjà

Ce qui est prêt	Où	Ce que ça évite à l'intégrateur
Lexique partagé des occasions tunisiennes et correspondance vers les contextes de M2 (`work` / `casual` / `event`)	`pipeline/context.py` (`OCCASIONS` , `M2_OCCASION`)	BO1 et M14 peuvent réutiliser la même liste
Journal d'événements au format `Event` partagé (`local_search` , `product_view` , `shop_click`)	`events.py`	Le tableau de bord M14 et les analyses lisent un seul format
LLM optionnel avec repli sur les règles ; même serveur Ollama que l'orchestrateur possible	`llm.py`	Sans Ollama, tout marche ; pas de clé d'API
Tests isolés : dossiers temporaires, LLM éteint, date figée, aucun appel réseau ; ruff propre	`tests/conftest.py` (M7 et M14)	`pytest` passe sur n'importe quelle machine en ~40 s
Données dans git (catalogue nettoyé, index FashionCLIP, photos) ; sans données, `mock.py` répond	`ml/data/shops/`	Rien à télécharger ni DVC à configurer
Démarrage rapide : FashionCLIP chargé au lancement de l'API (`warm_up`) ; `serve_app.py` sert l'app et l'API sur un seul port, sans CORS, en IPv4 et IPv6	`router.py` , `mobile/tool/serve_app.py`	Première recherche sans attente de 13 s ; pas de lenteur de 2 s sur `localhost`

1. Où est le code

```
backend/modules/m07_local_catalogue/   M7 (agent localshop)
backend/modules/m14_stylist_mode/      M14
mobile/lib/features/bo6_local_market/  écran « Boutiques locales » (M7)
mobile/lib/features/bo6_boutique/      « Espace pro » + page « Aider un client » (M14)
mobile/tool/serve_app.py               sert l'app web + l'API sur un seul port (pas de CORS)
ml/data/shops/                         catalogue, photos (327 Mo), index FashionCLIP : versionnés dans git
ml/data/bo6/, ml/data/logs/            mémoire, données M14, journaux : JAMAIS dans git (recréés à l'exécution)
rapport/                               rapports M7 et M14 (md + pdf)
```

Fichiers partagés modifiés par BO6 (attention aux conflits, §5) : `mobile/lib/app/router.dart` (3 routes ajoutées) et `mobile/tool/serve_app.py` (transmet `/m07/` et `/m14/` , écoute en IPv4 et IPv6, connexions gardées ouvertes). Rien dans `backend/shared/` ni dans `backend/orchestrator/` .

2. M7 — Catalogue local (agent `localshop`)

2.1 Ce qu'il fait

`"baskets blanches à Tunis max 80 TND"` (français, anglais, darija, arabe) → compréhension de la requête → recherche hybride (FashionCLIP texte→photo + BM25) → filtres (prix, ville, type, couleur, genre, stock) → score + diversité → résultats avec boutique, prix en TND, photo, lien et raisons.

2.2 Endpoints (`/m07`)

Endpoint	Rôle
`POST /m07/search`	Corps `LocalShopInput`, réponse `LocalShopOutput` (+ `search_id`)
`POST /m07/outfits`	Tenues avec la garde-robe autour d'une pièce locale (idée 3) : `{user_id, product_id, occasion?, n}`
`POST /m07/local-pieces`	Pièces locales pour une tenue du jour (idée 4) : `{user_id, item_ids, occasion?, season?, city?, max_price_tnd?}`
`POST /m07/try-on` · `GET /m07/try-on/{id}.glb`	Une pièce locale en 3D pour l'avatar (idée 6, service du BO9) : `{product_id}` → `model_url`
`GET /m07/card` · `GET /m07/identity`	Fiche de l'agent (lue par l'orchestrateur)
`POST /m07/events`	Clics de l'app : `product_view`, `shop_click` (alimente le tableau de bord M14)
`POST /m07/feedback` · `GET /m07/history/{user}` · `DELETE /m07/memory/{user}`	Mémoire long terme, droit à l'oubli
`GET /m07/shops` · `GET /m07/products/{id}` · `GET /m07/images/{name}`	Boutiques, un produit, photo locale

2.3 Contrat (champs utiles à l'intégration)

- **Entrée** : `query` et/ou `gap` / `category` / `subcategory` / `image_embedding` (512, FashionCLIP) ; filtres `city`, `max_price_tnd`, `min_price_tnd`, `color`, `gender` ; `user_id`, `style_vector` (id M5 ou 512 valeurs) ; `k`, `offset`.
- **Occasion et saison (optionnels)** : `occasion` (mariage, fiançailles, henna, aïd, ramadan, entretien, soutenance, travail, soirée, plage, ou un mot du lexique comme « wedding ») et `season` (`summer` / `winter`) ; sinon lus dans le texte. Dans `parsed` : `occasion`, `season`, `occasion_pieces` et `usual_budget_tnd` (budget habituel appris par la mémoire, il ne fait que reclasser).
- **Champs agent (tous optionnels)** : `session_id`, `language` (`fr` / `en` / `ar`), `answer` (`{question_key, value}` = réponse à la question précédente), `next_page`.
- **Sortie** : `results[]`, `parsed`, `total_matches`, `note`, `search_id`, et pour l'orchestrateur : `agent`, `decision_id`, `question` + `question_key`, `memory_notes`, `next_actions[]`.
- `next_actions` : `ask_user`, `open_shop`, `judge` → **smartbuy**, `show_outfits` → **stylist**, `try_on` → **tryon** (BO9), `more_results` → localshop, `feedback`.
- **Garde-robe** (si `user_id` est donné) : `results[i].wardrobe_matches` et `sources` (M1 / M2, §2.3 bis).

- **Exemple dans l'app** (écran « Local shops », client de démo `usr_042`) : « jean » → carte « The agent asks: For women or for men? » ; un clic envoie `answer` dans la même `session_id` (130 → 79 jeans femme) et la réponse est gardée en mémoire (« Forget my answers » = `DELETE /m07/memory/{user}`). Dans la fiche d'un produit : « Talk to a stylist » crée un vrai hand-off M14 ; « Is it a good buy? » montre ce qui est passé à Smart Buy (`judge`), dont l'écran BO5 n'est pas encore fusionné. C'est ce que l'assistant BO8 doit faire avec `question` et `next_actions`.

2.3 bis Idées en simulation qui deviennent réelles à l'intégration

Ces idées du BO6 sont **codées et testées**, mais elles s'appuient sur des modules d'autres équipes. Tant que leurs branches ne sont pas fusionnées, une **simulation** répond à leur place (mêmes formats d'entrée et de sortie). Chaque réponse dit d'où viennent les données : `sources: {"m1": "mock", "m2": "mock"}`. **À l'intégration, la simulation devient réelle** sans toucher au code de M7 :

Idée BO6	Ce que fait M7	Module appelé	En simulation aujourd'hui	Pour passer au réel
Selected 10 · « Why this local piece »	Chaque résultat d'un utilisateur dit avec quelles pièces de **sa garde-robe** il va (`wardrobe_matches` + raison « Va avec ta garde-robe : … »)	M1 `POST /m01/items` + M2 `POST /m02/score`	Garde-robe fixe de 12 pièces, compatibilité par une règle (types complémentaires, couleurs neutres)	Fusionner `garde-robe` (M1) et `feat/m02-m03-m04-m17` (M2), puis `real: [m1, m2]` dans `m07_local_catalogue/config.yaml`
Selected 3 · Local piece → complete outfit	`POST /m07/outfits` : des tenues complètes avec la garde-robe autour d'une pièce de boutique, avec « pourquoi cette pièce »	M1 + M2 `POST /m02/complete` (pièce épinglée)	Tenues assemblées par la même règle	Même chose : `real: [m1, m2]`
Selected 4 · Outfits featuring Tunisian brands	`POST /m07/local-pieces` : pour une tenue du jour (`item_ids` de la garde-robe), une pièce locale qui la **complète** (chaussures, sac, veste…) ou **remplace** une pièce, notée par M2 avec toute la tenue	M1 + M2 `POST /m02/score`	Même règle de compatibilité	`real: [m1, m2]` **et** le BO1 appelle `/m07/local-pieces` depuis son écran « tenue du jour »
Backup 4 · Looks tagged with local brands	M14 envoie chaque look à la carte de partage avec la boutique de chaque pièce (`brand`)	M10 `POST /m10/cards` (BO7)	Pas de vraie carte (`card_url` et `share_url` vides)	Fusionner `feat/bo7-m10-m15`, puis `real: [m7, m10]` dans `m14_stylist_mode/config.yaml`

À vérifier après la fusion : - `POST /m01/items {user_id}` renvoie `{"items": [Garment…]}` avec leur `embedding` (vecteur M2 de 1024 valeurs) ; - M2 accepte la pièce de boutique **en ligne** dans `payload.items` (elle n'est pas dans la garde-robe) : M7 l'envoie au format `Garment` avec son vecteur [image FashionCLIP | texte

FashionCLIP] ; - un identifiant utilisateur commun à M1 et M7 (voir §4.1) ; - `pytest modules/m07_local_catalogue/tests/test_wardrobe.py` : le chemin « réel » y est déjà testé avec un faux M1 / M2, et une panne retombe sur la simulation.

Smart Buy (BO5) n'est pas appelé directement par M7 : c'est un agent, et un agent n'en appelle pas un autre. L'action suivante `judge` laisse l'orchestrateur (BO8) le chaîner.

2.3 ter Les 3 idées qui dépendaient d'un autre BO : comment elles sont réalisées

Même principe que §2.3 bis : le BO6 fait sa partie maintenant, l'autre module est **simulé avec son vrai format**, et à l'intégration on ajoute ce module à `real: [...]`.

Selected 6 · Essai 3D des pièces locales (BO9) - *Comment* : `POST /m07/try-on {product_id}` prend la photo de la pièce (copie locale du catalogue) et l'envoie au service 3D du BO9 : `POST http://127.0.0.1:8766/reconstruct` (TripoSR, `ai/triposr_server.py` sur la branche `feat/bo9-avatar-3d`), qui renvoie un **modèle 3D (GLB)**. M7 le garde en cache (`m1/data/bo6/tryon/` , jamais dans git) et le sert à `GET /m07/try-on/<id>.glb` (`model_url`) pour la visionneuse de l'avatar. L'action suivante `try_on` (outil `tryon`) contient `endpoint: "/m07/try-on"` . - *Aujourd'hui* : simulé → `model_url: null` , `sources.m9 = "mock"` , une note le dit ; l'app affiche « Simulation: BO9 3D service not connected yet ». - *À l'intégration* : lancer le service TripoSR du BO9 (port 8766), puis `real: [m9]` dans `m07_local_catalogue/config.yaml` (réglages `tryon.url` , `tryon.resolution` , `tryon.timeout_s`). Vérifier : `POST /m07/try-on` → `model_url` rempli, `sources.m9 = "real"` , le GLB se télécharge. Le BO9 affiche ce GLB sur l'avatar (Three.js / `avatar_studio`). Compter ~1 minute par pièce sur processeur (une seule fois, ensuite le cache répond).

Backup 2 · Codes fidélité des boutiques locales (BO6 + M13) - *Comment* : l'accord avec une boutique, c'est son **compte boutique de l'Espace pro (M14)**. La boutique y met son code : `PATCH /m14/pro/{id} {"loyalty": {"code": "MANUELLA10", "discount_pct": 10, "until": "2026-12-31", "conditions": "dès 100 TND"}}` (ou la carte « Loyalty code » de l'onglet Clients de l'app : code, %, date de fin, conditions ; la liste « Another shop » ouvre le compte de n'importe laquelle des 43 boutiques du catalogue) ; `DELETE /m14/pro/{id}/loyalty` l'arrête. M7 l'affiche sur **toutes les pièces de cette boutique** (`results[i].loyalty` + raison « Code fidélité MANUELLA10 : −10 % »). `GET /m14/loyalty` donne les codes valides (date `until` non passée). - *Aujourd'hui* : **réel côté BO6**. Les 4 comptes de démo ont des codes fictifs marqués `-DEMO10` (`simulate/pros.py`) : aucun vrai accord. - *À l'intégration* : M13 (BO5, buy links) lit `GET /m14/loyalty` et met le `promo_code` de la boutique sur ses liens (aujourd'hui M13 met `promo_code: null` pour les boutiques du catalogue M7). Pour la vraie vie : supprimer les codes `-DEMO10` et laisser chaque boutique saisir le sien.

Backup 5 · Défis de marques locales (M15, BO7) - *Comment* : un compte boutique sponsorise un défi : `POST /m14/challenges {pro_id, duration_days: 7, min_local_pieces: 1, no_repeat: true}` . **Un seul défi actif par boutique** : un second renvoie 409 tant que le premier n'est pas fini (`ends_on` passé) ou terminé par `POST /m14/challenges/{id}/end` (bouton « End challenge »). Chaque défi a `status` (`active` / `ended`), `ends_on` et `ended_at` . M14 construit le `ChallengeDraft` de M15 (règle M15 `local_brand` : au moins une pièce de marque locale par jour, + `no_repeat` , `sponsor` = la boutique), le publie sur `POST /m15/challenges/create` et ajoute 6 pièces de la boutique (M7) à suggérer aux participants. `GET /m14/challenges?pro_id=` liste les défis de la boutique. - *Aujourd'hui* : M15 simulé → `challenge_id` commence par `sim_chl_` , `sources.m15 = "mock"` ; l'app (onglet Clients, compte boutique) affiche « Simulation: M15 not connected yet ». - *À l'intégration* : fusionner `feat/bo7-m10-m15` , puis `real: [..., m15]` dans `m14_stylist_mode/config.yaml` . Vérifier : `challenge_id` donné par M15, le défi apparaît dans l'onglet Challenges du BO7. Pour que la règle `local_brand` fonctionne, les pièces achetées dans une boutique locale doivent avoir `local_tag` dans la garde-robe (M1) : M7 donne le nom de la boutique (`shop`). M15 n'a pas

encore de route pour **clore** un défi : aujourd'hui « End » le termine côté BO6 et M15 le clôt à la fin de `duration_days`. Proposition au BO7 : `POST /m15/challenges/{id}/end` ; M14 l'appellera depuis `service.end_challenge` (même schéma `clients._try` que la création). - *Comment le client le voit* : **dans l'écran Challenges du BO7**, pas dans l'app BO6 (l'Espace pro sert à la boutique). Sur la branche `feat/bo7-m10-m15` : - liste (`bo7_sharing/ui/challenges_screen.dart`) : une carte avec le titre (« 7 days with Manuella »), la description, « N joined » et l'icône boutique + **Manuella** (champ `sponsor`, déjà envoyé par M14) ; - détail (`challenge_detail_screen.dart`) : « Sponsored by Manuella », les règles (`local_brand` : au moins une pièce de marque tunisienne par jour, `no_repeat`), le bouton « Join · 7 days », puis les onglets My progress (la tenue du jour, vérifiée par M15), Leaderboard et Posts. - *Ce que le BO7 doit ajouter à l'intégration* : une section **« Pieces from Manuella »** dans le détail du défi. Le `ChallengeDraft` de M15 n'a pas de champ pour des pièces, donc elles restent dans M14 : `GET /m14/challenges/{challenge_id}` (même identifiant que M15) renvoie `pieces` (jusqu'à 6 pièces de la boutique : `item`, `price_tnd`, `image_path` servi par M7 à `GET /m07/{image_path}`, `product_url`), `status` et `ends_on`. Afficher seulement si `sponsor` est rempli ; 404 = défi qui ne vient pas d'une boutique, ne rien afficher. Chaque pièce ouvre `product_url` (site de la boutique) et peut envoyer `POST /m07/events {type: "shop_click"}` pour que la boutique voie ce clic dans son tableau de bord. Afficher aussi le code fidélité de la boutique s'il existe (`GET /m14/loyalty?shop=Manuella`).

Tests : `m07_local_catalogue/tests/test_last_ideas.py` (4) et `m14_stylist_mode/tests/test_local_brand.py` (6) ; le chemin réel est testé avec un faux BO9 et un faux M15.

2.4 Les 16 idées du BO6 : état, code et intégration

Légende : ✅ fait et réel · 🟡 fait, **simulé** jusqu'à la fusion d'un autre BO (la simulation devient réelle à l'intégration) · ❌ pas fait. **Les 16 idées ont maintenant leur partie BO6 faite.**

Selected (10)

#	Idée	État	Où dans le code	Ce qu'il faut à l'intégration
1	Curated Tunisian boutiques and brands	✅	M7 : 43 boutiques, 6 375 pièces (`scraper/` , `ml/data/shops/`), `POST /m07/search` , `GET /m07/shops`	Rien. Relancer le scraper de temps en temps (les prix changent)
2	Smart Buy suggesting local pieces	✅ (côté BO6)	M7 accepte `gap` (« white shirt ») ; Smart Buy (BO5) appelle M7 pour ses `buy_links`	Fusionner `feat/bo5-smartbuy` ; vérifier que Smart Buy appelle bien `/m07/search` avec `gap`
3	Local piece photo → complete outfit	🟡	`POST /m07/outfits` (M1 + M2 `/complete`) ; une photo → `image_embedding` dans `/m07/search`	`real: [m1, m2]` dans `m07_local_catalogue/config.yaml` (§2.3 bis)
4	Outfits featuring Tunisian brands	🟡	`POST /m07/local-pieces` (tenue du jour → pièce locale qui complète ou remplace, M2 `/score`) ; looks M14 avec pièces locales	`real: [m1, m2]` **et** le BO1 affiche les suggestions dans son écran « tenue du jour » en appelant `/m07/local-pieces` avec les `item_ids` de la tenue
5	Assistant: “where in Tunis can I find…?”	✅	Outil `localshop` (`tool.py` , carte `identity.py`), FR / EN / darija / arabe, question + mémoire + `next_actions`	Fusionner `feat/bo8-orchestrator` ; l'assistant doit poser `question` et suivre `next_actions` (§2.3)

#	Idée	État	Où dans le code	Ce qu'il faut à l'intégration
6	3D try-on of local pieces	🟡	`POST /m07/try-on` : photo → modèle 3D (GLB) par le service TripoSR du BO9, servi à `GET /m07/try-on/{id}.glb` ; action `try_on` ; bouton « Try it on my avatar »	Lancer le service du BO9, puis `real: [m9]` dans M7 ; le BO9 affiche le GLB sur l'avatar (§2.3 ter)
7	Tunisian-aware clothing categories	✅ (catalogue)	`taxonomy.py` : jebba, kachabia, sefsari, hijab, tenues traditionnelles ; abaya, caftan, qamis disent aussi le genre	M1 (garde-robe) doit utiliser les mêmes codes de catégories pour les pièces de l'utilisateur
8	Outfits for Tunisian occasions and seasons	✅	`pipeline/context.py` : mariage, henné, Aïd, Ramadan, plage… ; été / hiver ; partagé avec M14	Rien
9	Budget-aware suggestions in dinars	✅	Budget en TND dans la requête ; budget habituel appris (`memory.budget_samples` , `agent._usual_budget`)	Rien. Plus tard : lire aussi les achats réels (feedback `bought_product_id`)
10	"Why this local piece" explanation	🟡	`reasons` de chaque résultat (type, couleur, budget, ville, marque tunisienne, saison) + « Va avec ta garde-robe » (M1 + M2)	`real: [m1, m2]` (§2.3 bis)

Backup (6)

#	Idée	État	Où dans le code	Ce qu'il faut à l'intégration
1	Direct links to Tunisian online shops	✅	Chaque résultat a `product_url` ; action `open_shop` ; clics journalisés (`POST /m07/events`)	Rien (les partenariats et codes sont l'idée Backup 2)
2	Local boutique loyalty codes	✅ (côté BO6)	Code saisi par la boutique dans son compte M14 (`PATCH /m14/pro/{id}`), affiché par M7 sur ses pièces ; `GET /m14/loyalty`	M13 (BO5) lit `GET /m14/loyalty` pour ses liens ; remplacer les codes `-DEMO10` par de vrais codes (§2.3 ter)
3	Boutique-owner dashboard of local demand	✅	M14 `GET /m14/dashboard` + conseils `GET /m14/dashboard/insights`	Comptes boutique réels ; données réelles au lieu du simulateur (§4)
4	Looks tagged with local brands	🟡	Chaque look M14 garde la boutique de chaque pièce et l'envoie à M10 (`clients.share_card` , champ `brand`)	Fusionner `feat/bo7-m10-m15` , puis `real: [m7, m10]` dans `m14_stylist_mode/config.yaml`
5	Local brand style challenges	🟡	`POST /m14/challenges` : la boutique sponsorise un défi (règle M15 `local_brand`) avec ses pièces suggérées	Fusionner `feat/bo7-m10-m15` , puis `real: [..., m15]` dans M14 (§2.3 ter)
6	Aggregated local trend insights	✅	M14 tendances (z-score) et thèmes (FashionCLIP + k-means) à partir des recherches anonymes	Données réelles ; plus tard, les garde-robes agrégées (M1) comme le dit l'idée

2.5 Qui appelle M7 aujourd'hui

- **Orchestrateur** : `tool.py` (nom `localshop`, description construite depuis `identity.py`).
- **Smart Buy (BO5)** : en process `m7.service.search(body, channel="smartbuy")` ou en HTTP `POST /m07/search`. Les champs ajoutés sont optionnels, donc rien n'est cassé.
- **BO7** (`feat/bo7-integration`) : pièces des boutiques locales pour les looks.

2.5 bis Améliorations de qualité (24/10)

Point	Ce qui a changé	Mesure
Genre estimé sur la photo	Mots qui disent le genre (abaya, caftan, burkini → femme ; qamis → homme) ; seuil 2,0 au lieu de 1,5 ; une estimation contraire au type est écartée (robes, jupes, escarpins, lingerie : ≥ 95 % « femme » selon les boutiques)	98,2 % juste (contre 96,7 %), sur 1 955 produits dont le genre est écrit (`ml/data/shops/attributes_report.md`)
Saison sur la photo	Si le nom ne dit pas la saison, FashionCLIP compare la photo à « summer … » / « winter … » (`pipeline/attributes.py` , colonnes `season` / `season_source`)	86 % juste sur 577 produits dont le nom dit la saison ; 3 480 produits sur 6 375 ont maintenant une saison (1 620 avant, par le nom et le type)
Occasion habillée	Mariage, Aïd, entretien… : les pièces de sport, de plage ou d'intérieur (baskets, shorts, « veste sport », jogging…) sont écartées, sauf si on demande ce type	Plus de « veste sport » pour un mariage
Fraîcheur du catalogue	Si les pages des boutiques ont été lues il y a plus de `stale_after_days` (30 j), chaque réponse le dit dans `note`	—
Arabe	Notes, raisons, noms des pièces et des couleurs en arabe (`language: "ar"`) ; mots arabes des pièces dans les requêtes (قميص، فستان، تنورة، حذاء، حقيبة…)	—
Garde-robe simulée	12 pièces au lieu de 8 (deux bas, deux paires, une jupe, un manteau) : plusieurs tenues possibles	—

Après un nouveau scraping, relancer dans l'ordre (README de M7) : `scraper.run` → `scraper.clean` → `pipeline.ingest` → `pipeline.attributes` (genre et saison). Le nettoyage écrase les colonnes de saison ; les attributs les recalculent.

2.6 À savoir pour faire tourner M7

- **Données** : `ml/data/shops/` est dans git (CSV nettoyé, index, photos). Rien à télécharger.
- **Modèle** : FashionCLIP (`patrickjohncyh/fashion-clip` , environ 600 Mo) se télécharge depuis Hugging Face la première fois, puis se charge **au démarrage de l'API** (`warm_up: true` , ~13 s) : la première recherche n'attend pas. On peut aussi le placer dans `ml/models/fashion-clip/` .
- **Dépendances** : `backend/modules/m07_local_catalogue/requirements.txt` (torch, transformers).
- **Sans données** (clone sans le catalogue), le service répond avec `mock.py` , sans planter.

---

3. M14 — Stylist / boutique mode

3.1 Fonctions et endpoints (`/m14`)

Fonction	Endpoints	Pour
F1 Comptes pro	`POST/GET /m14/pro` , `GET/PATCH /m14/pro/{id}`	tous
F2 Consentement + fiche	`POST /m14/clients` , `POST /m14/clients/{link}/consent` , `DELETE /m14/clients/{link}` , `GET /m14/clients` , `GET /m14/clients/{client}/brief` , `GET /m14/clients/{client}/wardrobe`	BO4 BO5 BO7
F2b Conversation privée	`GET/POST/DELETE /m14/chat/{pro}/{client}`	BO6 BO8
F3 Relecture du profil	`POST /m14/clients/{client}/review` , `GET /m14/reviews/agreement`	BO4
F4 Looks	`POST/GET /m14/looks`	BO7
F5 Vendeur en magasin	`POST /m14/instore/session`	BO5
F6 Tableau de bord	`GET /m14/dashboard?shop=&days=&source=` , `GET /m14/pro/{id}/dashboard`	BO6
F7 Hand-off	`POST /m14/handoff/check` , `POST /m14/handoff` , `GET /m14/handoff/queue` , `GET/PATCH /m14/handoff/{id}`	BO8
Codes fidélité (Backup 2)	`PATCH /m14/pro/{id}` (`loyalty`), `DELETE /m14/pro/{id}/loyalty` , `GET /m14/loyalty`	BO5 (M13), M7
Défis de marques locales (Backup 5)	`POST/GET /m14/challenges` , `GET /m14/challenges/{id}` (pièces pour l'écran du BO7), `POST /m14/challenges/{id}/end`	BO7 (M15)

Formes exactes : `contract.py` . Réponses fixes pour les tests des autres BO : `mock.py` .

3.2 Données de M14

- `ml/data/bo6/m14/*.json` : `pros` , `links` , `handoffs` , `looks` , `reviews` , `chats` . Interface `store.Collection` (`all / get / put / delete / find`).
- `ml/data/logs/m14_simulated_events.jsonl` : trafic simulé, séparé du vrai journal M7 (`ml/data/logs/m07_events.jsonl`). Le tableau de bord lit les deux et affiche la part simulée.
- Données de démo (non versionnées, à recréer) : `bash cd backend python -m modules.m14_stylist_mode.simulate.pros --reset # 8 stylistes, 4 boutiques, clients usr_042..044 python -m modules.m14_stylist_mode.simulate.traffic # 12 semaines de recherches via la vraie M7 (~2 min)`

3.3 Comment M14 appelle les autres modules

Tout passe par `clients.py` . Pour chaque module : appel réel s'il est listé dans `config.yaml → real` , sinon (ou en cas d'erreur) réponse mock. Chaque réponse de M14 indique `sources: {m1: real|mock, …}` .

Module	Appel fait par M14	Ce que M14 lit	Aujourd'hui
M7 (BO6)	en process : `m7.service.product/shops/memory()`	produits, boutiques, recherches du client	**réel**
M1 (BO3, `garde-robe`)	`POST /m01/items {user_id}`	`items[]` : `item_id`, `category`, `subcategory`, `colors[0].name`, `wear_count`, `price_tnd`, `image_cutout` / `image_raw`	mock
M3 (BO1/2)	`POST /m03/stats {user_id, payload: {item_ids}}`	`result.items[].wear_count`	mock
M5 (BO4)	`POST /m05/profile`, `/m05/gaps`, `/m05/feedback`	voir §4.4	mock (M5 n'existe pas)
M2 (BO1/2)	`POST /m02/score {user_id, payload: {items, outfits:[[ids]]}}`	`result.outfits[0].score`	mock
M6 (BO5)	`POST /m06/verdict {user_id, probe_image, price_tnd, shop_brand, language, session_id}`	`verdict`, `score`, `reasons`, `explanation`	mock
M10 (BO7)	`POST /m10/cards {user_id, kind:"outfit", caption, language, items: [CardItem]}`	`image_url`, `share_url`	mock
M15 (BO7)	`POST /m15/challenges/create {user_id, title, description, rules, duration_days, tags, sponsor}`	`challenge_id`, `participants`	mock

Les corps envoyés à **M2** et **M10** ont été validés contre leurs vrais contrats (branches `feat/m02-m03-m04-m17` et `feat/bo7-m10-m15`). Pour M2, les pièces sont envoyées **en ligne** (`payload.items`, format `Garment` partagé) : M2 ne note que les pièces qu'on lui donne, et les pièces de boutique ne sont pas dans la garde-robe du client.

---

4. Ce qu'il faut faire pour M14

4.1 Utilisateurs : un identifiant commun (à faire en premier)

- **Le `client_id` de M14 doit être le `user_id` de M1, M3, M5 et M7.** C'est ce qui relie la fiche, la garde-robe, l'historique de recherche et la conversation. Aujourd'hui, les clients de démo (`usr_042`, `usr_043`, `usr_044`) n'existent pas dans M1.
- Pour tester : créer dans M1 la garde-robe d'un client test avec le **même** `user_id`, ou utiliser un utilisateur de démo de M1 comme client dans M14.
- **Comptes pro** : un `pro_id` M14 doit être rattaché à un utilisateur (styliste ou propriétaire de boutique). À prévoir : `user_id` dans `Pro`, et le rôle `stylist` / `boutique` dans les comptes.
- **Authentification (JWT, `shared/config.py → jwt_secret`)**. Aujourd'hui, l'identité est **déclarée par l'appelant** dans ces paramètres ; ils devront venir du jeton :

Endpoint	Paramètre à remplacer par l'identité du jeton
`POST /m14/clients/{link}/consent`, `DELETE /m14/clients/{link}`	`client_id`
`GET/POST/DELETE /m14/chat/…`	`reader_id`, `sender_id`, `requester_id`
`GET /m14/clients/{client}/brief`, `/wardrobe`, `PATCH /m14/handoff/{id}`, `POST /m14/looks`, `/review`, `/instore`	`pro_id`

Les règles d'accès sont déjà codées dans `service.py` (`_require_consent`, `_chat_role`) : il suffit de leur passer l'identité authentifiée.

4.2 Vraie garde-robe (M1)

1. Fusionner la branche `garde-robe` (BO3, Amine).
2. Dans `backend/modules/m14_stylist_mode/config.yaml` : `real: [m7, m1]`.
3. Vérifier : la fiche affiche « Sources : M1 réel » ; `GET /m14/clients/{client}/wardrobe?pro_id=` renvoie les vraies pièces.
4. Amélioration app : la page « Aider » affiche les pièces en texte. Avec M1, chaque pièce a une photo (`image` dans `/wardrobe`) : l'afficher dans `_WardrobePicker` (`bo6_help_screen.dart`).

Rien d'autre à coder : le client M1 suit déjà le contrat `ItemsRequest` / `ItemsResponse`.

4.3 Port des vêtements (M3) et score des looks (M2)

- Fusionner `feat/m02-m03-m04-m17` ; `real: [m7, m1, m3, m2]`.
- M2 a besoin d'**au moins 2 pièces** dans le look, sinon M14 garde le score mock.
- Les pièces M7 sont converties au format `Garment` par `clients.product_as_garment` (photo = URL de la boutique, couleur + hex du catalogue). Si M2 a besoin du vecteur des pièces de boutique, M7 sait déjà le construire (`agent._garment` : [image FashionCLIP | texte FashionCLIP], 1024 valeurs, envoyé quand M2 est réel) : M14 peut réutiliser cette fonction.

4.4 Profil de style (M5, BO4) — à définir ensemble

M5 n'est pas encore implémenté. M14 attend :

Appel	Réponse attendue par M14
`POST /m05/profile {user_id}`	`result: {archetype, likes[], dislikes[], colours[], style_vector?[512]}`
`POST /m05/gaps {user_id}`	`result: {gaps: ["chaussures habillées", …]}` (textes ou objets avec `label`)
`POST /m05/feedback {user_id, payload: {source:"stylist", pro_id, corrections}}`	n'importe quelle réponse 2xx

Si BO4 choisit d'autres noms de champs, il suffit d'adapter `style_profile`, `style_gaps` et `send_corrections` dans `clients.py`. Le `style_vector` sert aussi au choix du styliste (F7) et à M7.

4.5 Vendeur en magasin (M6) et cartes (M10)

- `feat/bo5-smartbuy` → `real: […, m6]`. M14 envoie la **photo de la boutique en URL http** comme `probe_image` ; le contrat de M6 l'accepte (« data URL / http »). À vérifier en vrai avec Aziz.
- `feat/bo7-m10-m15` → `real: […, m10]`. Le format `CardItem` est validé ; la carte est créée à chaque look envoyé (`card_url` , `share_url` dans la réponse).
- **M13** (liens partenaires, BO5) n'est pas encore utilisé : quand il sera fusionné, transformer les `product_url` des looks en liens partenaires (`POST /m13/links`) dans `service.send_look` .

4.6 Orchestrateur (BO8) : brancher le hand-off sur le chat

Aujourd'hui, le hand-off se déclenche seulement depuis l'app (simulateur de l'Espace pro). À faire :

1. Dans `localshop` (`m07_local_catalogue/agent.py → _next_actions`), ajouter une action `{"action": "handoff", "label": "Parler à un styliste", "tool": None, "args": {"endpoint": "/m14/handoff"}}` quand la recherche ne donne rien ou que l'occasion est importante.
2. Côté orchestrateur, quand l'utilisateur choisit cette action (ou que `POST /m14/handoff/check` renvoie `escalate: true`), appeler : `json POST /m14/handoff {"user_id": "...", "session_id": "<thread_id>", "language": "fr", "messages": [{"role": "user", "text": "..."}, {"role": "assistant", "text": "..."}], "force": true}` Les messages viennent de `ChatState.messages` . La réponse contient `status` , la styliste choisie et le résumé. Si `already_open` vaut `true` , le client a déjà une demande en cours.
3. Afficher au client « Ines (styliste, Sousse) va vous répondre » et ouvrir la conversation privée.

4.7 Événements partagés

M7 écrit déjà `local_search` , `product_view` , `shop_click` au format du schéma `Event` partagé. À ajouter dans `shared/schemas/event.py` (PR relue par M6) : `local_search` , `product_view` , `shop_click` , `handoff_request` , `handoff_closed` , `look_shared` , `profile_review` .

4.8 Stockage : JSON → Postgres

`store.py` expose `Collection.all/get/put/delete/find` . Le remplacer par des tables (`pros` , `client_links` , `handoffs` , `looks` , `reviews` , `chat_messages`) **sans toucher à** `service.py` . Même chose pour la mémoire de M7 (`memory.py` , un JSON par utilisateur).

4.9 Application

- **Espace pro** (fait) : `/bo6_boutique?tab=0|1|2` , page `/bo6_boutique/help/:pro/:client?handoff=` .
- **À faire côté client** (app client, BO7/BO8) : accepter ou refuser une invitation, voir et retirer les accès, recevoir les looks (`GET /m14/looks?client_id=`), conversation (`/m14/chat` en tant que client), bouton « Parler à un styliste ».
- **Adresse de l'API** : `--dart-define=API_URL=...` . En web, `tool/serve_app.py` sert l'app et l'API sur le même port (transmet `/m07/` et `/m14/` , GET/POST/PATCH/DELETE). Si l'app web est servie ailleurs, ajouter `CORSMiddleware` dans `backend/api/main.py` (absent aujourd'hui).

4.10 Confidentialité (déjà en place, à garder)

- Pas de données client sans consentement (403) ; le client voit et retire les accès.

- Conversation privée : la styliste et le client seulement ; fermée si l'accès est retiré ; effaçable par le client.
- Tableau de bord : agrégats seulement, aucun identifiant, groupes de moins de 10 recherches masqués.
- Ce qui manque : `DELETE` de toutes les données M14 d'un client (demandes, looks), comme `DELETE /m07/memory/{user}`.

4.11 Le LLM du BO6 : Qwen 2.5 7B en local (Ollama)

Un seul LLM pour l'agent BO6 (M7 et sa compétence M14), dans `backend/modules/m07_local_catalogue/llm.py`. Il **lit et rédige** ; il ne choisit **jamais** un produit, un prix, un score ou un styliste.

Où	Ce que fait le LLM	Ce que le code vérifie
M7 `pipeline/understand.py`	Requête que les règles ne comprennent pas du tout → type de pièce, couleur, genre, budget, ville (JSON) + reformulation en français ; puis la recherche normale	Schéma JSON avec les codes du catalogue ; budget écrit dans la requête, ville et genre dits dans la requête ; gardé seulement si la recherche trouve des pièces
M14 résumé du hand-off	Une phrase claire pour la styliste	≤ 260 caractères, aucun chiffre que le client n'a pas écrit ; les lignes de faits restent calculées
M14 escalade	Conversations longues que les règles n'escaladent pas	Le LLM doit citer les mots du client ; une citation absente de la conversation est ignorée ; il peut ajouter un signal, jamais en retirer
M14 conseils boutique (`GET /m14/dashboard/insights`)	2-3 conseils à partir des chiffres du tableau de bord	Tout chiffre d'un conseil doit exister dans les chiffres, sinon le conseil est retiré ; conseils par règles si le LLM est éteint

Installation (machine qui fait tourner l'API) :

```
# 1. installer Ollama (https://ollama.com), puis :
ollama pull qwen2.5:7b          # ~4,7 Go
ollama serve                    # si le service n'est pas déjà lancé (port 11434)
```

Réglages : `backend/modules/m07_local_catalogue/config.yaml → llm` (`enabled` , `url` , `model` , `timeout_s` , un interrupteur par usage). Pour une machine faible : `model: qwen2.5:3b` (plus rapide, un peu moins juste, voir `evaluation/RESULTS_LLM.md`).

Langue des réponses : M7 répond dans la langue demandée (`language` : `fr` par défaut, `en` , `ar`) ; ses questions, notes, raisons, noms de pièces et de couleurs existent en français, anglais et arabe. M14 écrit en anglais, comme l'Espace pro de l'app. La requête du client peut toujours être en français, anglais, darija ou arabe. L'app Flutter BO6 est en anglais et envoie `language: "en"` à M7.

Sans Ollama, rien ne casse : chaque usage retombe sur les règles (même logique que réel/mock). Les tests n'appellent jamais Ollama (faux LLM).

Lien avec `shared/llm.py` (BO8) : l'orchestrateur passe par LangChain (`get_chat_model()`), le BO6 appelle directement l'API d'Ollama pour utiliser les sorties JSON contraintes par un schéma, sans dépendance de plus. Les deux peuvent utiliser le **même serveur Ollama**. Si l'équipe passe à un LLM par API, seule la méthode `LLM._chat` de `llm.py` est à adapter.

Temps de réponse mesurés sur processeur seul (i7-1165G7, 32 Go, sans GPU), avec `qwen2.5:7b` :

Usage	Temps	Avec `qwen2.5:3b`
Compréhension M7 (médiane sur 16 requêtes)	≈ 15 s	≈ 7,6 s
Résumé du hand-off	7-10 s	environ deux fois moins
Escalade (conversation longue)	9-14 s	environ deux fois moins
Conseils boutique	≈ 30 s	environ deux fois moins
Premier appel (chargement du modèle)	+ 15-20 s	+ 5-10 s

Résultat M7 (`evaluation/RESULTS_LLM.md`) : sur 16 requêtes que les règles ne comprennent pas, le type de pièce passe de **0/16 à 11/16** avec le 7B (12/16 avec le 3B), sans budget ni ville inventés. C'est pour cela que le LLM n'est appelé que lorsque les règles échouent, et que les conseils boutique sont générés à la demande (bouton « Générer » dans l'app), pas à chaque ouverture du tableau de bord.

---

5. Fusion : ordre conseillé et conflits attendus

1. **`BO6/local_catalogue` en premier** : M7 est utilisé par Smart Buy, BO7 et l'orchestrateur.
2. Puis `garde-robe` (M1), `feat/m02-m03-m04-m17` , `feat/bo5-smartbuy` , `feat/bo7-m10-m15` , `feat/bo8-orchestrator` .
3. Après chaque fusion, ajouter le module dans `real:` de M7 et/ou de M14 et relancer les tests (§5.1).

Conflits attendus - `feat/bo8-orchestrator` contient une **ancienne copie** de `backend/modules/m07_local_catalogue/` et un `m14_stylist_mode/identity.py` + une route `/identity` . **Garder la version BO6** de ces deux dossiers (elle contient la sienne : même nom `localshop` , mêmes compétences, même `handoff`) : `bash git checkout BO6/local_catalogue -- backend/modules/m07_local_catalogue backend/modules/m14_stylist_mode` - `mobile/lib/app/router.dart` : garder les routes de tout le monde (BO6 ajoute `/bo6_boutique` et `/bo6_boutique/help/...`). - `shared/agent_card.py` (branche BO8) : `identity.py` de M7 l'utilise s'il existe, sinon garde son propre texte. Rien à faire.

5.1 Checklist : après chaque fusion

Branche fusionnée	À modifier	Comment vérifier
`garde-robe` (M1, BO3)	`real: [m1]` dans **M7** ; `real: [m7, m1]` dans **M14**	`POST /m07/outfits` → `sources.m1 = "real"` ; fiche client M14 → « Sources : M1 réel »
`feat/m02-m03-m04-m17` (M2, M3)	`real: [m1, m2]` dans **M7** ; `[m7, m1, m3, m2]` dans **M14**	`POST /m07/local-pieces` → `sources.m2 = "real"` , scores différents de 0,8 ; looks M14 notés par M2

Branche fusionnée	À modifier	Comment vérifier
`feat/bo5-smartbuy` (M6, M13)	`real: […, m6]` dans **M14** ; M13 lit `GET /m14/loyalty` pour mettre le code de la boutique sur ses liens	Vendeur en magasin (`POST /m14/instore/session`) → verdict réel ; Smart Buy affiche des `buy_links` venant de M7, avec le code fidélité de la boutique
`feat/bo7-m10-m15` (M10, M15)	`real: […, m10, m15]` dans **M14**	Look envoyé → `card_url` et `share_url` remplis ; `POST /m14/challenges` → `challenge_id` donné par M15, `sources.m15 = "real"`
`feat/bo8-orchestrator`	garder les dossiers M7/M14 du BO6 (§5)	`localshop` apparaît dans les outils ; une `question` de M7 est posée au client ; `next_actions` suivies
`feat/bo9-avatar-3d` (BO9)	lancer `ai/triposr_server.py` (port 8766), puis `real: [..., m9]` dans **M7**	`POST /m07/try-on` → `model_url` rempli, `sources.m9 = "real"` ; l'action `try_on` (outil `tryon`) pose le GLB sur l'avatar
BO1 (écran « tenue du jour »)	appeler `POST /m07/local-pieces` avec les `item_ids` de la tenue	des pièces tunisiennes s'affichent sous la tenue du jour

Après chaque ligne : `pytest modules/m07_local_catalogue modules/m14_stylist_mode` (les tests forcent la simulation, ils restent verts) puis un essai dans l'app (§6).

5 bis. Tester avec de vrais utilisateurs (à faire avant la soutenance si possible)

Aujourd'hui les chiffres du tableau de bord viennent surtout du simulateur (93 %), la garde-robe est simulée et les 16 demandes du test LLM ont été écrites par l'équipe. Un petit test avec de vraies personnes donne des chiffres honnêtes, sans code en plus : chaque recherche et chaque clic sont déjà journalisés (`ml/data/logs/m07_events.jsonl` , résumé : `python -m modules.m07_local_catalogue.events --days 7`).

1. **5 à 10 personnes** (amis, famille), chacune avec son identifiant (`usr_…`), sur l'écran « Local shops ».
2. **5 tâches chacune** : une pièce précise (« baskets blanches moins de 100 dt »), une demande en darija, une occasion (« tenue pour un mariage »), une saison (« robe d'été »), une demande libre ; puis « Outfits with my wardrobe » et « Find Tunisian pieces » une fois.
3. **Après chaque tâche, une question** : « as-tu trouvé une pièce que tu achèterais ? » (oui / non) — le bouton feedback de M7 (`POST /m07/feedback`) l'enregistre.
4. **Mesurer** : part des recherches avec au moins un clic, part des « oui », demandes non comprises (`understood_by` vide et 0 résultat), temps de réponse. Écrire les nouvelles demandes difficiles dans `evaluation/queries_llm.yaml` : le test LLM n'est plus écrit seulement par l'équipe.

6. Vérifier que tout marche

```
cd backend
pytest modules/m07_local_catalogue modules/m14_stylist_mode orchestrator   # 249 tests sur la branche BO6
python -m modules.m14_stylist_mode.simulate.pros --reset
python -m modules.m14_stylist_mode.simulate.traffic
uvicorn api.main:app --port 8000                                           # http://localhost:8000/docs

cd ../mobile
flutter build web --dart-define=API_URL=http://localhost:8090
python tool/serve_app.py --api http://127.0.0.1:8000                       # http://localhost:8090/#/bo6_boutique
```

Contrôles rapides après chaque intégration : - `GET /m14/clients/usr_042/brief?pro_id=<pro>` → `sources` montre quels modules sont réels. - Espace pro → Hand-off → « Envoyer à l'assistant » → la demande arrive chez la bonne styliste → Accepter → page Aider → envoyer un look → il apparaît dans la conversation, avec un score M2. - Tableau de bord → une boutique → la part simulée est affichée en bandeau.

7. Réglages

7.1 M7 (`backend/modules/m07_local_catalogue/config.yaml`)

Clé	Rôle
`real` , `api_url` , `timeout_s`	modules appelés pour de vrai (`[]` aujourd'hui : M1 et M2 simulés) et adresse de l'API
`weights.*`	poids du score : visuel, texte, attributs, style, saison, budget habituel
`wardrobe.*`	compatibilité avec la garde-robe sur les résultats (idée 10) : nombre de résultats, seuil M2
`local_pieces.*`	pièces locales pour une tenue (idée 4) : candidats par emplacement, seuil M2
`budget_memory.*`	budget habituel (idée 9) : nombre minimal de budgets, fenêtre, tolérance
`gender_photo_margin` , `gender_type_prior` , `season_photo_margin`	seuils du genre et de la saison lus sur la photo (`attributes_report.md`)
`stale_after_days`	âge du catalogue au-delà duquel les réponses disent de vérifier les prix
`warm_up`	charger le catalogue et FashionCLIP au démarrage de l'API
`llm.*`	Qwen 2.5 7B : activé, modèle, délai, un interrupteur par usage (§4.11)
`memory_*`	mémoire de l'agent : dossier, historique, sessions
`tryon.*`	service 3D du BO9 : adresse, résolution, délai, dossier du cache GLB (idée 6)

7.2 M14 (`backend/modules/m14_stylist_mode/config.yaml`)

Clé	Rôle
`real`	modules appelés pour de vrai (`[m7]` aujourd'hui)
`api_url` , `timeout_s`	adresse de l'API pour les appels HTTP vers les autres modules
`min_group`	taille minimale d'un groupe affiché dans le tableau de bord (confidentialité)
`dashboard.*`	période, sources, nombre de thèmes, seuils de tendance, couverture du stock
`handoff.*`	seuil et poids de l'escalade, poids du choix du styliste, charge maximale
`challenge_pieces`	nombre de pièces de la boutique suggérées dans un défi de marque locale (Backup 5)
`dashboard.themes.cache_minutes`	durée pendant laquelle les thèmes du tableau de bord sont gardés (10 min) ; ensuite ils sont recalculés en arrière-plan ; 0 = toujours recalculés

8. Contacts

Module	Branche	Personne
M7, M14 (BO6)	`BO6/local_catalogue`	kenzabokri
M1 Garde-robe (BO3)	`garde-robe`	aminebali
M2, M3, M4, M17 (BO1/BO2)	`feat/m02-m03-m04-m17`	Raecm
M6, M13, M18 (BO5)	`feat/bo5-smartbuy`	Mohamed Aziz Chouaia
M10, M15 (BO7)	`feat/bo7-m10-m15`	Omar Ferchichi
M8, orchestrateur (BO8)	`feat/bo8-orchestrator`	Nadhir Maamar
M5 (BO4)	— (pas encore de branche)	à confirmer

Documentation détaillée : `backend/modules/m07_local_catalogue/README.md` , `backend/modules/m14_stylist_mode/README.md` , `rapport/rapport_M7_catalogue_local_FR.pdf` , `rapport/rapport_M14_stylist_boutique_FR.pdf` .